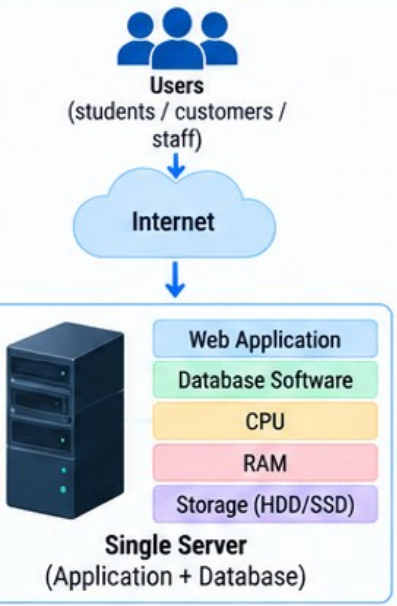
. Single Server – Concept and Working

hat is a Single Server?

single server is a system in which an application and its database n on one physical or virtual machine. The same server provides e required CPU, memory, storage and database services.

A single-server database system stores and processes data on one server that provides the required computing resources.

chitecture of a Single Server



How It Works

1. User sends a request (e.g., view marks).
2. Application receives the request.
3. Database is queried on the same server.
4. Database returns the data.
5. Application displays the result to the user.

mponents of a Server

CPU
cesses
tabase
erations

RAM
Holds frequently used data

Storage
Stores the database permanently

Network
Communicates with users and applications

DBMS
Manages the data

itable For

- Small applications
- Limited number of users
- Low to moderate data volume
- Simple deployment and management
- Useful for educational institutions, small businesses, etc.

2. Example and Scaling a Single Server

Example: Small E-Commerce Website

A small online store with 500 customers, 2,000 products and 100 orders per day can run on a single server.



Data Stored on the Server

- Customers
- Products
- Orders
- Payments
- Inventory

Example: College Management System

A college with 10,000 students can use a single server to store:



Scaling Up the Single Server (Vertical Scaling)

When the application grows, we can make the existing server more powerful.



The architecture is still a single server, but the server's capacity increases.

When to Move Beyond a Single Server?

A single server may become insufficient when:

- Number of users increases significantly.
- Data volume becomes very large.
- Request and transaction rates increase.
- Application requires higher availability.
- Hardware upgrades become too expensive or reach physical limits.
- You need to handle global or large-scale traffic.

3. Advantages, Limitations and Evolutio

Advantages of a Single Server

- Simple architecture
- Easy to configure and manage
- Lower initial cost
- Easy database transactions
- Suitable for small applications
- Good performance for moderate workloads
- Fewer machines to monitor and maintain

Limitations of a Single Server

- Limited scalability
- Hardware has physical limits
- Cost increases for very powerful machines
- Single point of failure (if the server fails, the application may be unavaila
- Difficult to handle extremely high traffic
- One machine can become a bottleneck

Single Point of Failure



If the single server fails, the application and database may become unavailable.

From Single Server to Cluster

Single Server (Initial Stage)



Works well for small applications.

Cluster (As the Application Grows



Add more servers to handle higher loa
(horizontal scaling).

Start with a single server for small applications, and move to a cluster when the application grows.

1. Understanding Sharding

hat is Sharding?

arding is the process of partitioning a large dataset into smaller eces called shards and distributing those shards across multiple rvers. Each shard contains a subset of the data.

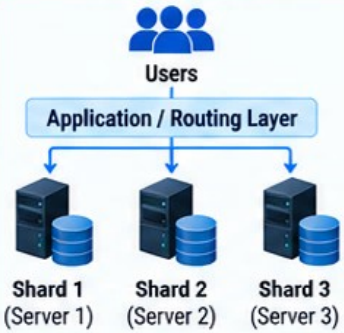
Sharding helps handle large amounts of data and high traffic by spreading the workload across multiple machines.

hy is Sharding Needed?

Without Sharding (Single Server)



With Sharding (Multiple Servers)



- All data stored on one server
- Becomes a bottleneck as data grows
- Limited storage, CPU and memory
- Not suitable for large applications

- Data is divided into smaller parts
- Each part (shard) stored on different server
- Load is distributed across servers
- Can scale by adding more servers

ard vs Replication

Sharding
different data on each server)



Data is divided into parts (shards) and stored on different servers.

Replication
(same data on multiple servers)



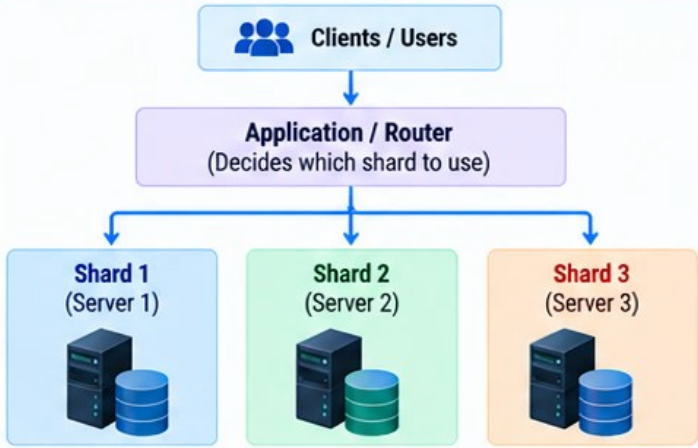
The same data is copied to multiple servers.

Key Takeaway

Sharding = Divide the data | Replication = Copy the data

2. How Sharding Works

Basic Architecture



How a Record is Located (Example)

1. Application receives a request for a record (e.g., Customer ID = C450).
2. Sharding logic (based on shard key) determines which shard contains the data.
3. The request is sent to the appropriate server (e.g., Server 2).
4. The server returns the requested data.
5. Application sends the result to the user.

Example: Range-Based Sharding

Shard	Customer ID Range	Server
Shard 1	C001 – C300	Server 1
Shard 2	C301 – C600	Server 2
Shard 3	C601 – C900	Server 3

Example: Hash-Based Sharding

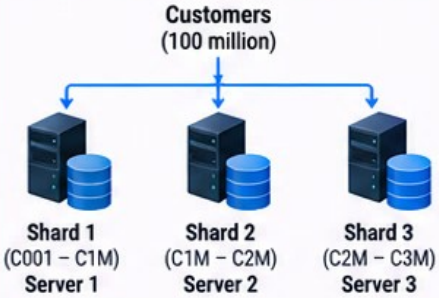
Shard = hash(CustomerID) % NumberOfShards

Customer ID	hash(ID)	hash(ID) % 3	Shard
C101	25	1	Shard 1
C205	17	2	Shard 2
C309	30	0	Shard 3

A shard key (e.g., CustomerID, UserID, AccountID, Region) is used to determine where a record should be stored.

3. Real-World Examples and Key Points

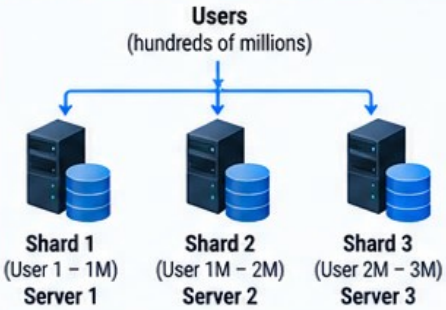
Example 1: E-Commerce Database



Use Case and Benefits

- Large number of customers orders and products.
- Shard by CustomerID, OrderID or region.
- Distributes storage and work
- Handles high traffic and gro
- New shards can be added e as the business grows.

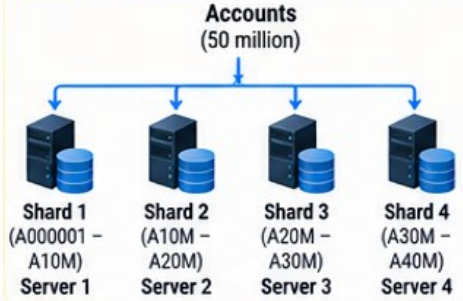
Example 2: Social Media Platform



Use Case and Benefits

- Store user profiles, posts, comments, likes, followers,
- Shard by UserID.
- Distributes read/write load across servers.
- Supports massive user bas and real-time activity.
- Improves performance and scalability.

Example 3: Banking / Financial Application



Use Case and Benefits

- Store account details, transactions and linked da
- Shard by AccountID or reg
- Distributes large financial
- Supports high transaction volume.
- Must be carefully designe for operations spanning multiple accounts.

Advantages of Sharding

- Horizontal scalability (add more servers)
- Handles very large datasets
- Distributes workload
- Reduces pressure on individual servers
- Suitable for high-traffic applications

Challenges / Limitations

- Increased system complexity
- Cross-shard queries can be difficul
- Distributed transactions are compl
- Choosing the right shard key is impo
- Possible hot shards (uneven load)
- Rebalancing shards when data gro

Sharding enables large-scale, high-performance applications, but it requires careful design and management.

1. How Aggregates are Handled in Key-Value vs Document Models

n **aggregate** is a collection of related data that is treated as a single unit for storage nd access (e.g., an order with customer details, line items and payment information).

Key-Value Model



Stores the aggregate as a value associated with a unique key.

The value is usually treated as an opaque unit by the database.

The complete aggregate is retrieved using the key.

Internal structure is handled by the application.

Simple and fast key-based access.

Document Model



- Stores the aggregate as a structured document.
- The database understands the internal structure of the document.
- Fields, nested objects and arrays are supported.
- Allows queries on individual fields.
- Provides richer access to the contents of the aggregate.

Both models store related data as a single unit, but they differ in how the database treats and accesses that data.

2. Example: Order Aggregate in a Key-Value Database

In a Key-Value database, the entire order (customer, line items and payment) can be stored as the value for a single key. The database treats the value as an opaque object.

Key

Order:1001

Value (Entire Aggregate)

```
{
  "orderId": 1001,
  "orderDate": "2026-09-05",
  "customer": {
     "id": 101,
     "name": "Ravi",
     "email": "ravi@example.com",
     "phone": "9876543210"
  },
  "items": [
    { "product": "Laptop", "quantity": 1, "price": 50000 },
    { "product": "Mouse", "quantity": 2, "price": 3000 }
  ],
  "payment": {
     "paymentId": 901,
     "method": "Credit Card",
     "amount": 56000,
     "status": "Successful"
  }
}
```

Key Points

- Accessed using a unique (e.g., Order:1001).
- The entire aggregate is retrieved at once.
- The database does not need to understand the internal structure.
- Updates usually involve storing the complete va (or using specific featur of the database).
- Best suited when the application already kno the key and needs the whole aggregate.

3. Example: Order Aggregate in a Document Database

a Document database, the order is stored as a structured document. The database understands e internal structure and can query individual fields.

Order Document (e.g., in MongoDB)

```
"order_id": 1001,
"order_date": "2026-09-05",
"customer": {
   "id": 101,
   "name": "Ravi",
   "email": "ravi@example.com",
   "phone": "9876543210",
   "address": "Hyderabad"
},
"items": [
  { "product": "Laptop", "quantity": 1, "price": 50000 },
  { "product": "Mouse", "quantity": 2, "price": 3000 }
],
"payment": {
   "payment_id": 901,
   "method": "Credit Card",
   "amount": 56000,
   "status": "Successful"
}
```

Example Queries

- Find an order by ID:
  `db.orders.find({ order_id: 1001 })`
- Find orders for a customer:
  `db.orders.find({ "customer.name": "Ravi" })`
- Find orders containing a product:
  `db.orders.find({ "items.product": "Laptop" })`

Key Points

- Stored as a single, structured document.
- Supports nested objects and arrays.
- The database understands the structure.
- Allows queries on individual fields.
- Flexible schema (different orders can have different fields).
- Better for complex and evolving data.

4. Comparison and Key Takeaways

Feature	Key-Value Model	Document Model
How aggregates are stored	As a value associated with a key (usually opaque)	As a structured document
Database understanding of structure	Generally does not understand the internal structure	Understands fields, nested objects and arrays
Access method	Mainly through a unique key	By key and by document fields (queries)
Query individual fields	Limited	Supported
Handling nested data	Possible, but treated as a single value	Naturally supported
Updates	Usually store the complete value	Can update entire document or specific fields
Use case suitability	Simple, fast lookups when key is known (e.g., session data, user preferences)	Complex, flexible data with rich query requirements (e.g., product catalog, orders)

Conclusion

Both Key-Value and Document models store related data as an aggregate, but the Key-Value model treats the value as an opaque unit accessed by a key, while the Document model stores a structured document that the database can understand and query. Choose the model based on the application's data complexity and access requirements.

1. What is Aggregate Orientation?

ggregate orientation means grouping related data and treating as a **single unit of data**, called an **aggregate**.

An aggregate is a collection of related data that is accessed or updated together.

Instead of splitting the data into many tables (as in relational databases), related information is stored together.

The goal is to keep data that belongs together in one logical unit so that the application can retrieve and update it as a whole.

Commonly used in document-oriented, key-value, and column-family NoSQL databases.

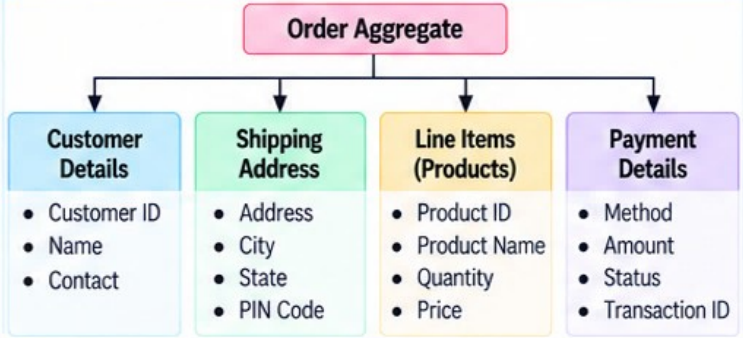
Key Points

- Related data is grouped as one unit.
- Designed around how the application accesses the data.
- Reduces the need for joins.
- Simplifies data access and can improve performance.
- Supports scalability in distributed environments.

2. Example: Order Aggregate in NoSQL

In an e-commerce application, an **order** can be an aggregate that contains all related information.

Structure of an Order Aggregate



Order Aggregate as a Document (e.g., in MongoDB)

```
{
  "order_id": 1001,
  "customer": { "customer_id": 101, "name": "Ravi" },
  "shipping_address": { "city": "Guntur", "state": "AP" },
  "items": [
    { "product": "Laptop", "quantity": 1, "price": 50000 },
    { "product": "Mouse", "quantity": 2, "price": 3000 }
  ],
  "payment": { "method": "Credit Card", "amount": 56000, "status": "Successful" }
}
```

3. Aggregate Orientation vs Relational Databases

Relational Database (Normalized)

Data is divided into multiple tables. Joins are required to get the complete information.

CUSTOMER

Customer_ID	Name	Email
101	Ravi	ravi@example.com

ORDERS

Order_ID	Customer_ID	Order_Date
1001	101	2025-09-01

ORDER_ITEMS

Order_ID	Product_ID	Quantity
1001	P1	1
1001	P2	2

PAYMENT

Order_ID	Amount	Status
1001	56000	Successful

Aggregate-Oriented NoSQL (Denormali

Related data is stored together as one document (aggregate). No joins are nee to retrieve the complete information.

```
{
  "order_id": 1001,
  "customer": {
    "customer_id": 101,
    "name": "Ravi",
    "email": "ravi@example.co
  },
  "items": [
    { "product_id": "P1",
      "name": "Laptop",
      "quantity": 1 }
    { "product_id": "P2",
      "name": "Mouse",
      "quantity": 2 }
  ],
  "payment": {
    "amount": 56000,
    "status": "Successful"
  }
}
```

4. Aggregate Boundaries and Design Considerations

hat to Include in an Aggregate?

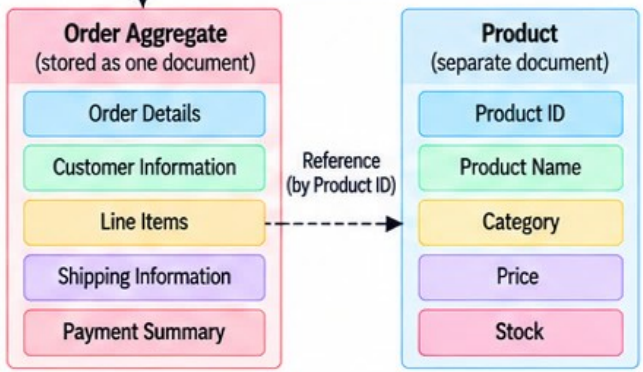
Data that is commonly accessed together.
Data that is updated together.
Information that naturally belongs to the same real-world entity.
Keep the aggregate size reasonable.
Avoid including data that is shared widely and changes independently.

xample

Order aggregate includes: customer details, items, shipping address, and payment details.

Product information (e.g., product name, category, current price) may be stored separately and referenced by product ID, because it is shared across many orders.

Example Design



5. Benefits and Limitations

Benefits of Aggregate Orientation

- Simpler data access (retrieve related data as one unit).
- Fewer joins compared to relational databases.
- Better performance for aggregate-level reads.
- Easier updates when related data changes together.
- Natural representation of real-world entities.
- Supports distributed storage and scalability.
- Flexible schema (especially in document databases).

Limitations and Challenges

- Some data may be duplicated (e.g., customer name in multiple orders).
- Very large aggregates can become inefficient.
- Cross-aggregate queries can be more complex.
- Updates to duplicated data may require changes in multiple aggregates.
- Designing correct aggregate boundaries can be diffic
- Not suitable for all types of queries (e.g., global reports across many aggregates).

Key Takeaway

Aggregate orientation is a practical way to model data in NoSQL databases by storing related information together as a single unit. It works well when the data is commonly accessed and updated together, but it require careful design to avoid large aggregates and data duplication.

e Aggregate-Oriented Databases?

ed databases are databases that store and manage es rather than as individual records. An **aggregate is** ated data that is treated as a single unit and **n a single key** (usually a unique identifier).

istics

aggregates: A group of related data is stored together

unique key: Each aggregate is retrieved using a single customerId).
updates: The entire aggregate is read, modified, and s a whole.
na: Different aggregates can have different fields.
r complex, real-world objects: Useful when data within naturally belongs together.
oSQL document databases: e.g., MongoDB, CouchDB.
calability: Aggregates can be distributed across ines.

2. Aggregate vs. Traditional Relational Mo

Relational Model (Normalized Tables)

Customer and **Orders** stored separately in different tables.

Customers Table

CustomerID	Name
1	Ravi
2	Sita
3	Kiran

Orders Table

OrderID	CustomerID	Product
101	1	Laptop
102	1	Mouse
103	2	Mobile

- Data is split across multiple tables.
- Relationships are maintained using foreign keys.
- To get a complete customer view, multiple queries and joins are required.

Aggregate-Oriented (One Document per A

Customer and all relate together as a single agg

```
{
    "customerId": 1,
    "name": "Ravi",
    "email": "ravi@ex
    "address":{
      "city": "Hyder
      "state": "TS"
    },
    "orders": [
      {   "orderId":
          "product":
          "amount":
      },
      {   "orderId":
          "product":
          "amount":

    ]
}
```

- Related data is stored a
- Accessed using a singl
- Simpler data retrieval fo

regate-Oriented Databases Work

cation



e an aggregate unique key stomerId)

Database



Stores aggregates as documents (e.g., JSON)

Aggregate (Document)

```
{
    "customerId": 1,
    "name": "Ravi",
    "email": "ravi@example.com",
    "orders": [...]
}
```

Key Ide

- Each aggregate is self-c
- The database does not nested records separate
- To update data, the app aggregate, modifies the and writes the whole ag
- This approach reduces is well-suited for distribu

rld Examples of Aggregate-Oriented Databases

merce (Customer Aggregate)

e can include customer details, list, and payment methods.

```
1001",
xample.com",
yawada",
9"
02001",
Mobile",
000,
elivered"
op",
Card"]
```

Why This Works Well?

- All customer-related information is stored in one place.
- Easy to fetch and display the complete profile.
- Simplifies application logic.
- Scales well for large numbers of customers.

Example 2: Banking/Finance (Account Aggregate)

An **account aggregate** can contain account details, transaction history, and linked cards.

```
{
   "accountId": "A12345",
   "customerId": "C5001",
   "accountType": "Savings",
   "balance": 45000,
   "transactions": [
      {
         "txnId": "T001",
         "type": "Deposit",
         "amount": 20000,
         "date": "2024-08-01"
      },
      {  "txnId":"T002",
         "type": "Withdrawal",
         "amount": 5000,
         "date": "2024-08-03"
      }
   ],
   "linkedCards": ["****1234",
   "****5678"]
}
```

Benefits

- Complete account information is stored together.
- Useful for generating account statements.
- Reduces the need for multiple queries.
- Works well in high-volume, distributed environments.

Example 3: Social Media (User Agg

A **user aggregate** can include profile inf followers, and settings.

```
{
  "userId": "U789",
  "name": "Kiran",
  "username": "kiran_tech",
  "profile": {
     "bio": "Learning NoSQL",
     "location": "Bangalore"
  },
  "posts": [
     {  "postId": "P1",
        "content": "Hello World!",
        "likes": 120,
        "comments": 15
     }
  ],
  "followers": 560,
  "following": 340,
  "settings": {
     "privacy": "public",
     "notifications": true
  }
}
```

gregate-Oriented Databases model data around real-world entities, storing related information together as a single, self-contained unit making them flexible, scalable, and well-suited for modern applications.

Combining Sharding and Replication – Concept

hat is it?

mbining sharding and replication means partitioning a large database o multiple shards and maintaining one or more replicas of each shard different nodes.

Sharding = Divide the data | Replication = Copy the data

Sharding + Replication = Scalable + Highly Available

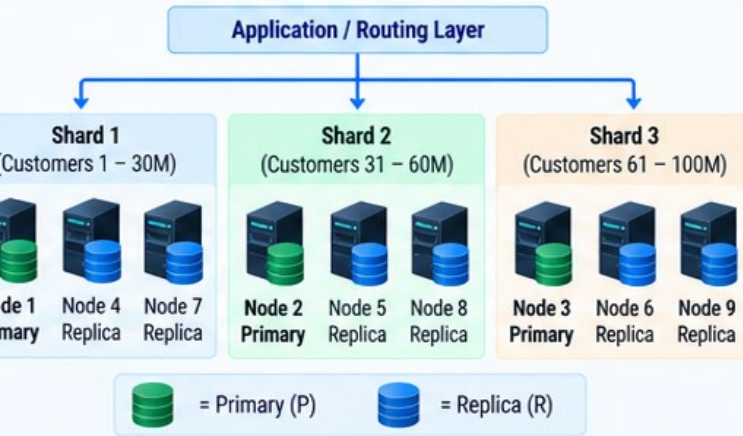
hy Combine Both?

- Sharding alone provides scalability but does not guarantee availability.
- Replication alone provides copies but does not solve the problem of storing a huge dataset on a single server.
- Combining both gives large-scale data distribution and multiple copies for fault tolerance.

fference Between Sharding and Replication

Feature	Sharding	Replication
rpose	Divide data	Create copies
ta on each server	Different subset	Same data (copy)
ain benefit	Scalability	Availability
orage usage	Distributed	Increased (multiple copies)
ult tolerance	Limited	High
orks well for	Large datasets	High availability

chitecture: Sharding + Replication



Each shard contains different data, and each shard has multiple copies on different nodes.

2. Detailed Example – E-Commerce Application

Use Case

A large online shopping application with 100 million customers and 500 million orders needs high scalability and high availability.

Step 1: Sharding the Data

Divide customer data based on CustomerID.

Shard 1	Shard 2	Shard 3
C1 – C30M customers	C31M – C60M customers	C61M – C100M customers
Customers 1 – 30M	Customers 31M – 60M	Customers 61M – 100M

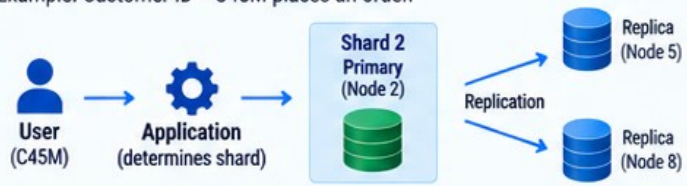
Step 2: Replicating Each Shard

Maintain multiple copies (e.g., 3 replicas) for each shard.

Shard 1 Replication Group	Shard 2 Replication Group	Shard 3 Replication Group
Node 1 P, Node 4 R, Node 7 R	Node 2 P, Node 5 R, Node 8 R	Node 3 P, Node 6 R, Node 9 R

Step 3: How a Write Operation Works

Example: Customer ID = C45M places an order.



1. Application receives the request.
2. Shard key (CustomerID) identifies Shard 2.
3. Data is written to the primary of Shard 2 (Node 2).
4. The change is replicated to other replicas (Node 5, Node 8).
5. Eventually all replicas contain the updated data.

Step 4: Handling a Server Failure

If the primary for Shard 2 (Node 2) fails, one of its replicas can be promoted as the new primary.



3. More Examples, Benefits and Key Poin

Example 1: Large E-Commerce Platform

- Shard by CustomerID or OrderID
- Each shard has multiple replicas
- Handles millions of users and orders
- Read traffic (product searches) can be served by replicas
- If a node fails, another replica can take over

100M custom
500M order

Example 2: Social Media Application

- Shard by UserID or region
- Replicate each shard on multiple nodes
- Users around the world can access nearby nodes
- Supports high availability and global scalability
- Continues operating even if some nodes fail

Hundreds of milli
of users

Example 3: IoT / Sensor Data System

- Shard by geographic region or deviceID
- Each shard has replicas
- Handles continuous sensor data from many devices
- Suitable for distributed and geographically spread systems
- Remains available even if one location goes down

Millions of sens
across locatio

Advantages and Disadvantages

Advantages

- Horizontal scalability
- High availability
- Fault tolerance
- Distributes read and write workload
- Supports very large datasets
- Read scalability using replicas
- Suitable for global applications
- Can continue operating even if individual nodes fail

Disadvantages

- Increased system complexity
- Storage overhead (multiple copies
- Network overhead
- Cross-shard queries can be compl
- Distributed transactions are difficu
- Conflict management may be requi
- Rebalancing when adding or remo nodes can be challenging
- Troubleshooting can be harder

Key Takeaways

1. Sharding divides the data. Replication creates copies.
2. Combining both provides scalability and high availability.
3. Each shard has its own replication group (primary + replicas).
4. The system can handle large datasets and continue operating even if some nodes
5. Careful design is required for routing, consistency, and conflict handling.

Divide, Copy, Distribute – Build Large, Reliable, and Scalable Systen

Distributed Data Models in NoSQL – Overview

hat is a Distributed Data Model?

istributed data model defines how data is stored across multiple vers (nodes) in a cluster. NoSQL databases are designed to scale out r many machines, and distribution is a key part of their architecture.

The two main techniques for distributing data are: Sharding and Replication.

hy Distribute Data?

- Handle very large datasets
- Support high read and write traffic
- Scale horizontally by adding more machines
- Provide high availability and fault tolerance
- Avoid the limits and high cost of very large single servers

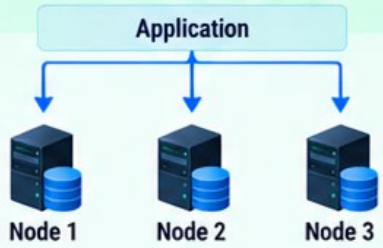
ngle Server vs Distributed Database

Single Server (Scale Up)



All data on one server
Limited scalability
Becomes expensive
Single point of failure

Distributed (Scale Out)



- Data distributed across multiple nodes
- Add more machines as data grows
- Higher scalability, availability and performance

mponents of a Distributed NoSQL System

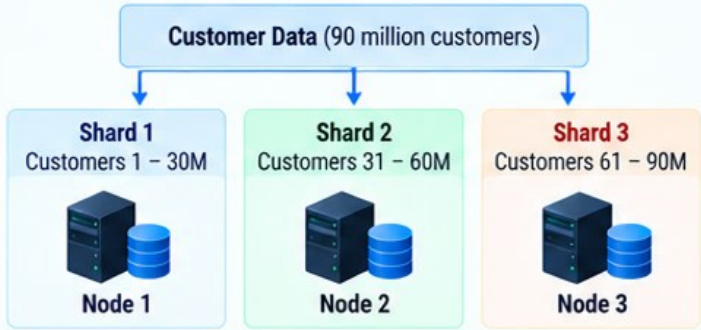
Data Model	How data is organized (e.g., aggregates).
Distribution Model	How data is spread across nodes (sharding, replication).
Consistency Model	How updates are propagated and seen by different nodes.
Cluster Management	Node membership, failure detection, rebalancing, etc.

Distribution allows NoSQL databases to handle large-scale data and high workloads using clusters of machines.

2. Sharding and Replication

Sharding – Divide the Data

Sharding splits a dataset into smaller parts (shards) and stores each part on a different server.

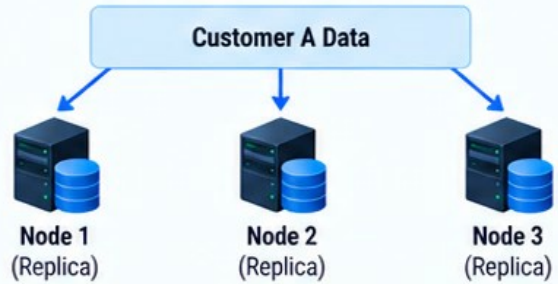


Key Points about Sharding

- Different data on different nodes
- Provides horizontal scalability
- Helps distribute read and write workloads
- Does not by itself provide fault tolerance

Replication – Copy the Data

Replication keeps multiple copies of the same data on different nodes.



Key Points about Replication

- Same data on multiple nodes
- Improves availability and fault tolerance
- Allows reads to be distributed across replicas
- Increases storage usage due to multiple copies

Sharding + Replication

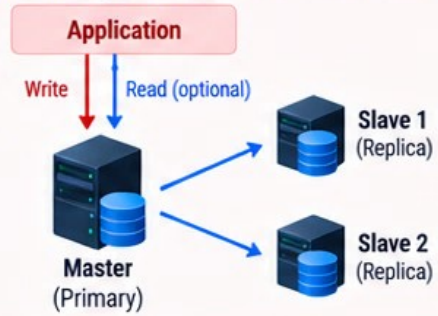
Both techniques can be combined to get scalability and availability.

Each shard can have multiple replicas.

3. Replication Models and Combined Examp

Master-Slave Replication

One node is the master (handles writes) and one or more slaves maintain copies and are often used for reads.

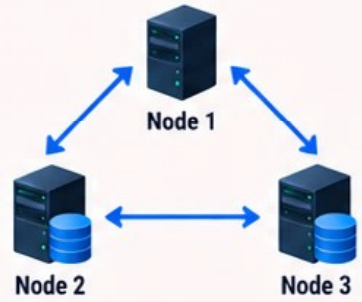


Characteristics

- Single master for writes
- Slaves replicate data fr master
- Simpler write coordinat
- Read scaling using slav
- Suitable for many use c

Peer-to-Peer Replication

All nodes are peers. Any node can accept writes, and nodes coordinate to keep their copies consistent.

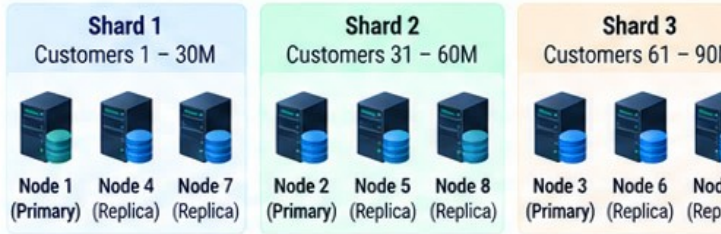


Characteristics

- No single master
- All nodes can accept writ
- Nodes coordinate to synchronize data
- More complex consisten handling
- Common in many NoSQL systems (e.g., column-fam databases)

Combining Sharding and Replication – Example

A large e-commerce system with 90 million customers, sharded into 3 parts, with 3 replicas per shard.



Benefits

- Scalability through sharding
- High availability through replication
- Can handle large datasets and high traffic
- System can continue operating even if some nodes fail.

1. Problem and Input Data

Goal: Using Map-Reduce, calculate the total quantity sold and total revenue for each product from a set of customer orders.

Customer Orders (Input Data)

Order ID	Product	Quantity	Price (₹)
1	Laptop	2	50,000
2	Mouse	3	1,000
3	Laptop	1	50,000
4	Keyboard	2	2,000
5	Mouse	2	1,000

Each order contains the product name, quantity and unit price.
Revenue for an order = Quantity × Price

What We Need to Find

- Total quantity sold for each product
- Total revenue for each product

Map Output Format

(Product, (Quantity, Revenue))

Reduce Output Format

(Product, (Total Quantity, Total Revenue))

2. Map Phase

Each Map task reads an order and generates a key-value pair using the product as the k and (quantity, revenue) as the value.

Each Order is Processed by Map

Order 1
(Laptop, 2, 50,000)

Order 2
(Mouse, 3, 1,000)

Order 3
(Laptop, 1, 50,000)

Order 4
(Keyboard, 2, 2,000)

Order 5
(Mouse, 2, 1,000)



Map Output (Intermediate Key-Value Pairs)

Product	(Quantity, Revenue)
Laptop	(2, 1,00,000)
Mouse	(3, 3,000)
Laptop	(1, 50,000)
Keyboard	(2, 4,000)
Mouse	(2, 2,000)

Revenue = Quantity × Price

3. Shuffle, Group and Reduce Phase

The system groups all values with the same product (key) and sends them to the same Reducer. Each Reducer calculates the total quantity and total revenue for its product.

Grouped Data (After Shuffle)

Product	List of (Quantity, Revenue)
Laptop	[(2, 1,00,000), (1, 50,000)]
Mouse	[(3, 3,000), (2, 2,000)]
Keyboard	[(2, 4,000)]

Reduce Tasks

- **Reducer 1** (processes Laptop)
- **Reducer 2** (processes Mouse)
- **Reducer 3** (processes Keyboard)

Reduce Function (Logic)

For each product:
total_quantity = sum of all quantities
total_revenue = sum of all revenues
emit(product, (total_quantity, total_revenue))

Example: Laptop

Input values: [(2, 1,00,000), (1, 50,000)]
Total quantity = 2 + 1 = 3
Total revenue = 1,00,000 + 50,000 = 1,50,000
Output: (Laptop, (3, 1,50,000))

4. Final Output and MapReduce Flow

Final Reduce Output

Product	Total Quantity	Total Revenue (₹)
Laptop	3	1,50,000
Mouse	5	5,000
Keyboard	2	4,000

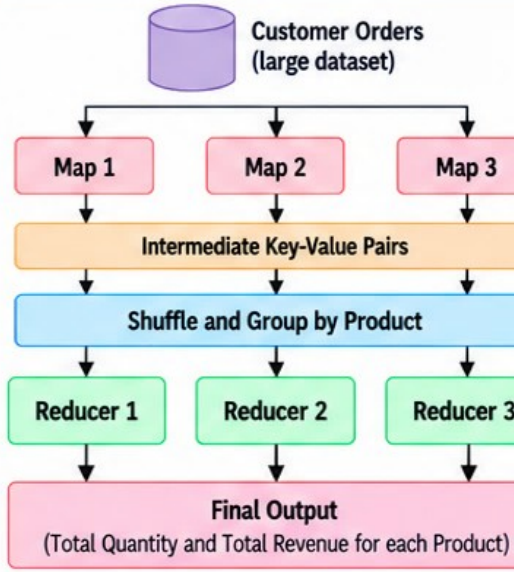
Interpretation

- Laptop: 3 units sold, total revenue ₹1,50,000
- Mouse: 5 units sold, total revenue ₹5,000
- Keyboard: 2 units sold, total revenue ₹4,000

Key Takeaways

- Map processes each order independently.
- All values for the same product go to the same Reducer.
- Reduce calculates the total quantity and revenue.
- Multiple Map and Reduce tasks can run in parallel for large datasets.

Overall MapReduce Flow



1. Map Output and Partitioning Idea

lap tasks process input data and generate **intermediate key-value pairs.** Partitioning ivides these key-value pairs and sends them to different **Reducers** based on the key, o that all values for the same key go to the same Reducer.

Map Output (Example)

Key	Value
CSE	80
ECE	75
CSE	90
ME	65
ECE	85
CSE	70
ME	80
ECE	90
CSE	95

Partitioning Function

partition(key) = hash(key) % R

R = number of Reducers

↓

hash(key) % 3
(for 3 Reducers)

The partitioner determines which Reducer should receive each key-value pair based on the key.

2. Distributing Key-Value Pairs to Reducers (Example)

Using the partitioning function, the intermediate key-value pairs are distributed among multiple Reducers. All values for the same key are sent to the same Reducer.

Map Output

Key	Value
CSE	80
ECE	75
CSE	90
ME	65
ECE	85
CSE	70
ME	80
ECE	90
CSE	95

→ **Partitioner** (hash(key) % 3)

Reducer 1 (CSE)
(CSE, 80)
(CSE, 90)
(CSE, 70)
(CSE, 95)

Reducer 2 (ECE)
(ECE, 75)
(ECE, 85)
(ECE, 90)

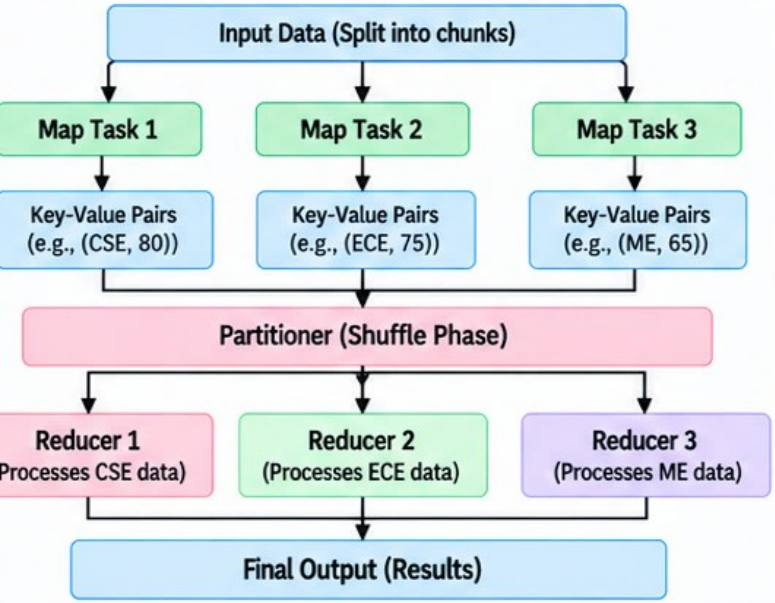
Reducer 3 (ME)
(ME, 65)
(ME, 80)

Key Points

- All values for the same key go to the same Reduce
- Different keys ca go to different Reducers.
- The data is distributed based on the partitionin function.
- This enables parallel processin in the Reduce pha

3. MapReduce Workflow with Partitioning

artitioning is part of the **Shuffle phase**. After Map tasks generate key-value pairs, the partitioner istributes them to Reducers. Each Reducer then processes its assigned data in parallel.



Shuffle and Sort

1. Map tasks generate key-value pairs.
2. Partitioner assigns each key to a Reducer.
3. Intermediate data is transferred (shuffle) and sorted by key.
4. Reducers process their assigned keys in parallel.

4. How Partitioning Improves Parallel Processing

Partitioning divides the workload among multiple Reducers, allowing them to process their data simultaneously. This reduces processing time and improves resource utilization.

Without Partitioning (One Reducer)

All key-value pairs go to a single Reducer.

90 million key-value pairs



Reducer 1

High load
Longer processing time
Single point of bottleneck

Using Partitioning →

With Partitioning (Three Reducers)

Key-value pairs are distributed among Reducers.

90 million key-value pairs



Reducer 1 (~30 million) | Reducer 2 (~30 million) | Reducer 3 (~30 million)

Reducers work in parallel
Lower processing time
Better resource utilization
Scales to larger datasets

Benefits

- Enables parallel processing
- Reduces overall processing time
- Distributes worklo evenly (if partition is well chosen)
- Improves resource utilization
- Supports scalabili with more Reduce
- Avoids single Redu bottleneck

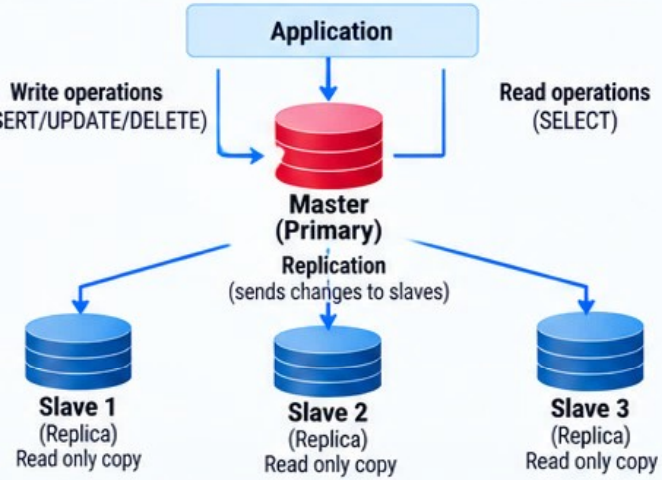
. Master-Slave Replication – Basics

hat is Master-Slave Replication?

ster-Slave Replication is a database replication model in which e server (master) handles write operations and one or more servers aves) maintain copies of the master's data. Slaves are commonly ed for read operations.

One primary copy + one or more replicated copies

sic Architecture



w It Works (Step by Step)

1. Application sends a write request to the master.
2. Master stores the change in its database.
3. Master sends the change to slave(s) through a replication mechanism.
4. Slaves receive the change and update their copies.
5. Application can read data from the slaves.

aster vs Slave Roles

Master (Primary)

Accepts all write operations
Maintains the main copy of data
Sends changes to slaves
Can become a bottleneck if write load is very high

Slave (Replica)

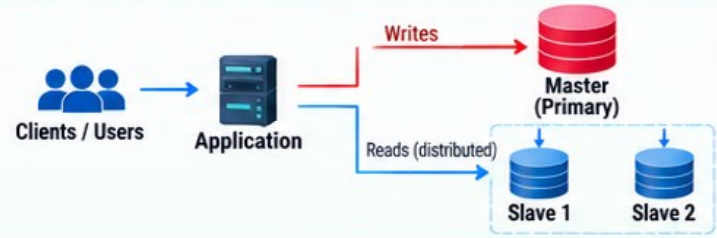
- Maintains a copy of the master's data
- Used for read operations
- Cannot usually accept writes
- Helps in load balancing, backup, and high availability

2. Key Concepts and Benefits

Replication Flow Example



Read and Write Flow



Why Use Master-Slave Replication?

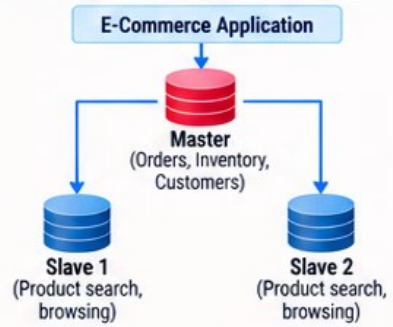
- Improves availability (data is copied to multiple servers)
- Distributes read load across replicas
- Provides backup copies of data
- Supports disaster recovery and failover
- Useful for read-heavy applications
- Replicas can be placed in different geographic locations

Important Concepts

Replication Lag	Slaves may receive updates slightly later than the master.
Eventual Consistency	Replicas will eventually contain the same data as the master (if no further updates occur).
Failover	If the master fails, a replica can be promoted to become the new master (manual or automatic).
Read Scaling	Multiple replicas can handle many read requests.
Write Bottleneck	All write operations go to the master, which can become a performance bottleneck.

3. Real-World Examples, Advantages and Limitatio

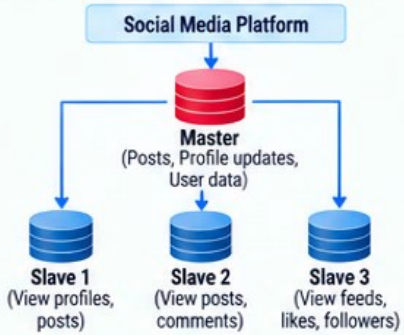
Example 1: E-Commerce Website



Scenario

- Handles millions of customers products.
- Master stores orders, inventory updates, and customer change
- Slaves serve product searches browsing, and reporting queries
- Helps in handling high read tra during sales and promotions.

Example 2: Social Media Application



Scenario

- Millions of users generate larg amounts of read traffic.
- Master handles writes such as new posts, comments, profile updates.
- Slaves serve read operations viewing posts, profiles, and comments.
- Provides scalability and bette user experience.

Example 3: News / Content Website



Scenario

- Writers publish and update artic (handled by master).
- Millions of readers access artic (handled by replicas).
- Read operations are much high than write operations.
- Replication helps in handling la read traffic and provides backu copies.

Advantages

- High availability
- Read scalability
- Fault tolerance
- Backup and reporting
- Geographic distribution
- Useful for read-heavy workloads

Limitations

- Single write bottleneck (master)
- Replication lag (slaves may be stale)
- Failover can be complex
- Not suitable for very high write workl
- Data consistency challenges
- Network dependency

Master-Slave Replication provides multiple copies for better availability and read scalability, but writes are concentrated on a single master.

. What is Peer-to-Peer Replication?

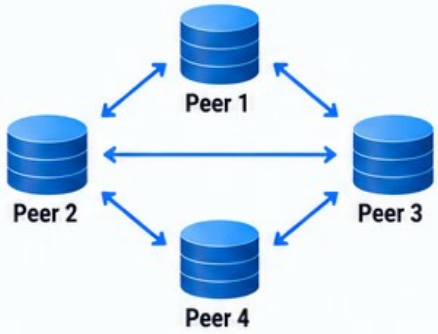
eer-to-Peer (P2P) Replication is a database replication model in which multiple database odes act as equal peers. There is no single permanent master. Each node can accept read nd write operations, and changes made on one node are replicated to other nodes.

Multiple nodes, equal participation, data is replicated among all nodes.

ey Characteristics

- No single permanent master.
- Multiple nodes act as peers.
- Each node can accept read and write operations.
- Data changes are replicated among participating nodes.
- High availability and fault tolerance.
- Supports horizontal scaling.
- Suitable for distributed and global applications.
- Consistency is often eventual.
- Conflict resolution may be required.

Peer-to-Peer Architecture

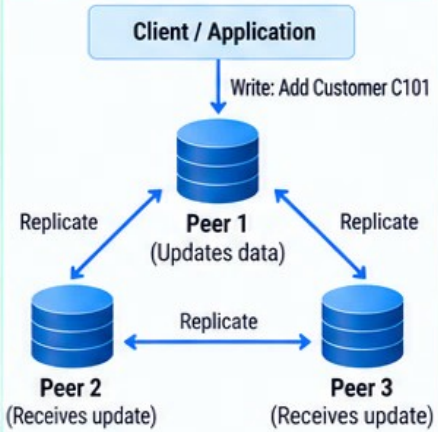


Peer-to-Peer = Team of equals (no single master)

2. How Peer-to-Peer Replication Works

1. A client sends a write request to any peer (e.g., Peer 1).
2. The peer updates its local data.
3. The change is replicated to other peers.
4. Other peers receive the change and update their copies.
5. All peers eventually have the same data (according to the system's consistency rules).

Example: Write Operation Flow



Data After Replication

Peer	Customer C10
Peer 1	Ravi
Peer 2	Ravi
Peer 3	Ravi

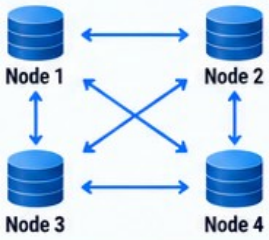
Read Operation

- Clients can read from any pe
- Read requests can be distribu across multiple nodes.
- Helps in handling large read workloads.

A write at one node is replicated to other nodes, and all peers eventually converge.

Real-World Examples

xample 1: Cassandra-Style istributed Database



y Points

- All nodes are peers.
- No single master.
- Handles very large datasets.
- Designed for high availability and horizontal scalability.
- Suitable for distributed applications.

ample technology: Apache Cassandra

Example 2: Global Social Media Application

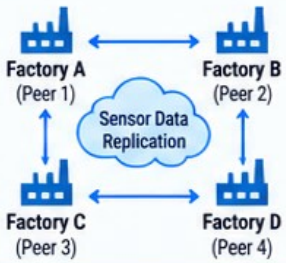


Key Points

- Multiple regional peers (no single master).
- Users can write and read from nearby nodes.
- Data (posts, comments, profiles) replicates across regions.
- Helps in handling massive read and write traffic.
- Improves user experience with lower latency.

Use case: Social media platforms

Example 3: Distributed IoT System



Key Points

- Each location can store and write sensor data.
- Data replicates among all peers.
- Can continue operating even if one node fails.
- Suitable for large volumes of time-series data.
- Helps in geographically distributed environments.

Use case: Industrial IoT / Smart Manufacturing

4. Advantages, Disadvantages and Comparison

Advantages

- ✓ No single permanent master (removes single point of failure).
- ✓ High availability and fault tolerance.
- ✓ Supports horizontal scaling (add more nodes).
- ✓ Distributes read and write workload.
- ✓ Suitable for geographically distributed systems.
- ✓ Can continue operating even if some nodes fail.
- ✓ Useful for large-scale, highly distributed applications.

Disadvantages

- ✗ Conflict resolution is required when multiple nodes update the same data.
- ✗ Data may be temporarily inconsistent (eventual consistency).
- ✗ More complex to manage and monitor.
- ✗ Network dependency (nodes must communicate)
- ✗ Troubleshooting can be more difficult.
- ✗ Replication overhead.
- ✗ Not ideal for applications requiring strong consistency for every operation.

Master-Slave vs Peer-to-Peer

Feature	Master-Slave Replication	Peer-to-Peer Replication
Primary node	One master	No permanent master (all peers)
Write operations	Handled by master	Multiple nodes can accept writes
Read operations	Handled by slaves (typically)	Handled by any node
Architecture	Hierarchical	Symmetric (peers)
Failure impact	Master failure requires failover	No single permanent primary
Conflict handling	Generally simpler	Can be more complex
Write scalability	Can be limited by master	Can distribute writes across nodes
Consistency	Often easier to reason about	Often eventual consistency
Best suited for	Moderate write workloads, simpler setups	Large, distributed, highly available systems

Peer-to-Peer Replication provides high availability and scalability, but requires careful handling of consistency and conflicts.

1. Problem and Solution

Scenario

A NoSQL database contains **90 million customer** records. Storing all records on a single server can lead to:

- High storage requirements
- High CPU and memory usage
- Increased read and write load
- Slower response time
- Limited scalability

Solution: Sharding

Use sharding to divide the large dataset into smaller portions (shards) and store them on multiple servers.

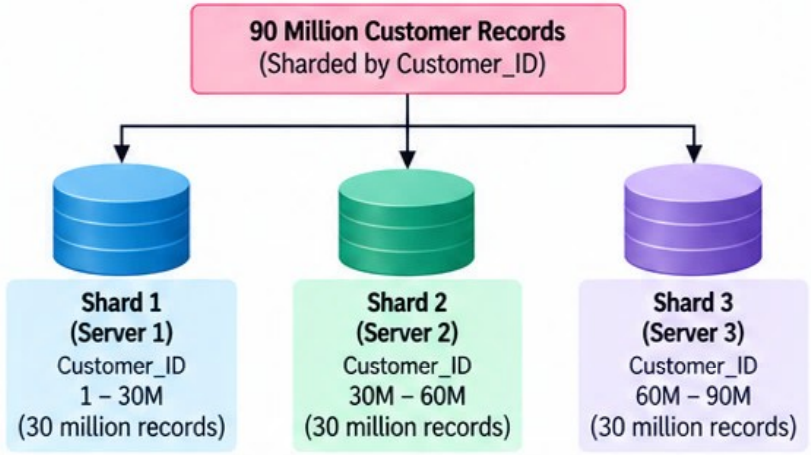
In this example, we use three servers, so the 90 million customer records are divided into three shards, with approximately 30 million records on each server.

Sharding improves performance, scalability, and allows the system to handle large amounts of data.

2. Dividing the Data (Sharding)

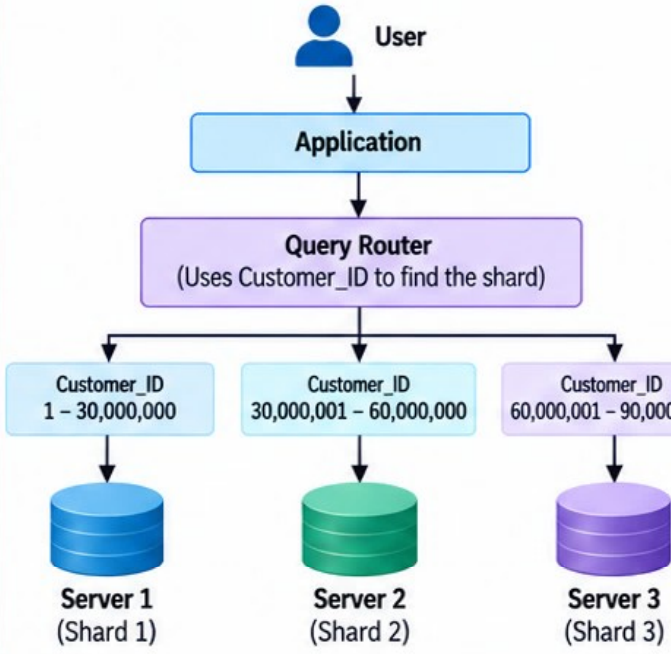
We use **Customer_ID** as the **shard key** and apply **range-based** sharding. The 90 million records are divided into three equal ranges.

Shard	Server	Customer ID Range	Approx. Records
Shard 1	Server 1	1 – 30,000,000	30 million
Shard 2	Server 2	30,000,001 – 60,000,000	30 million
Shard 3	Server 3	60,000,001 – 90,000,000	30 million



3. Routing User Requests

A query router (or routing mechanism) examines the Customer_ID in each request and determines which shard contains the required data. The request is then sent only to the relevant server.



4. Example Requests

Let's see how different customer requests are routed to the correct shard.

Example	Customer_ID	Range Check	Target Shard	Target Server	Action
1	15,000,000	1 – 30,000,000	Shard 1	Server 1	Fetch customer record
2	45,000,000	30,000,001 – 60,000,000	Shard 2	Server 2	Fetch customer record
3	75,000,000	60,000,001 – 90,000,000	Shard 3	Server 3	Fetch customer record
4	85,000,000	60,000,001 – 90,000,000	Shard 3	Server 3	Insert new customer record

Routing Flow for a Request (e.g., Customer_ID = 45,000,000)

User Request (Customer_ID = 45,000,000) → **Query Router** (Check range) → **Send to** Shard 2 (Server 2) → **Return** Customer Record

5. Benefits and Additional Considerations

How Sharding Improves Performance

- ✓ Distributes data across multiple servers
- ✓ Improves read performance (parallel processing)
- ✓ Improves write performance
- ✓ Reduces storage load on individual servers
- ✓ Supports horizontal scalability (add more shards)
- ✓ Handles large datasets efficiently
- ✓ Better resource utilization

Important Considerations

- ⚠ Choose a good shard key to avoid uneven data distribution.
- ⚠ Cross-shard queries can be more complex and may require querying multiple shards.
- ⚠ Adding or removing shards may require data rebalancing.
- ⚠ Operational complexity is higher compared to a single-server setup.
- ⚠ If data is not evenly distributed, some shards may become hot spots.

Summary

By using range-based sharding on **Customer_ID**, we can distribute **90 million** customer records across three servers (30 million records each). A query router directs each user request to the appropriate shard, enabling parallel processing, better scalability, and improved overall performance.

1. Scenario: High Reads, Low Writes

A NoSQL database is experiencing a very high number of read requests but relatively few write requests. This is common in applications like e-commerce, social media, content platforms, and product catalogs.

Example Workload

Read requests
100,000 per minute
(Very High)

Write requests
2,000 per minute
(Relatively Low)

Typical Read Operations

- View product details
- Search products
- View reviews
- Browse categories

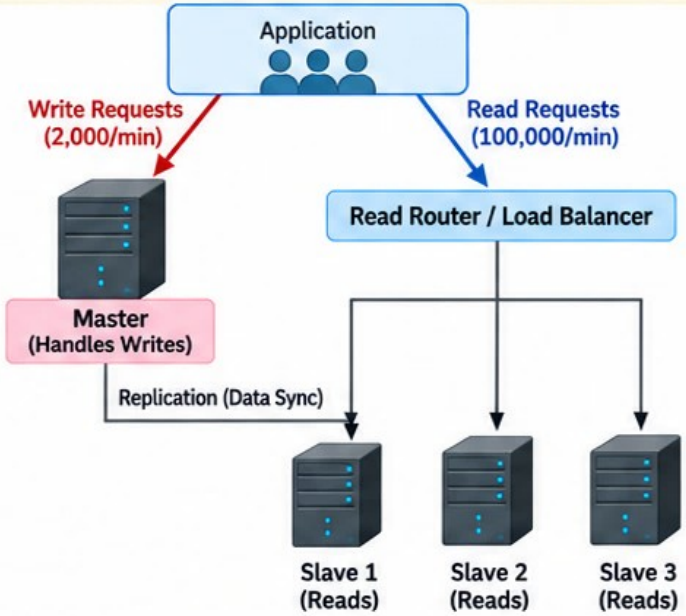
Typical Write Operations

- Add new product
- Update product price
- Update stock
- Create order

This is a read-heavy workload, so we can use Master-Slave Replication to distribute the load.

2. Master-Slave Replication Architecture

In Master-Slave Replication, the master handles all write operations and replicates the data to multiple slave nodes. The slave nodes handle read operations.



How it Works

1. All write requests go to the master.
2. The master replicates changes to slave nodes (synchronously or asynchronously).
3. Read requests are distribu across slave nodes using read router or load balan
4. Slaves serve the read reque in parallel.

3. How the Workload is Distributed

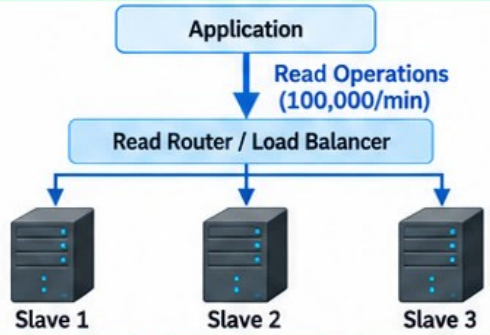
Write Path (to Master)



Write Operations Examples

- Insert new product
- Update product price
- Update stock
- Create order
- Modify customer information

Read Path (to Slaves)



Read Operations Examples

- View product details
- Search products
- View reviews
- Browse categories
- View order status
- Generate reports

4. Benefits and Important Considerations

Benefits of Master-Slave Replication

- Reduces load on the master – master handles only write operations.
- Improves read performance – multiple slaves serve reads in parallel.
- Provides horizontal read scaling – add more slaves when read traffic increases.
- Better availability – if one slave fails, others can continue serving reads.
- Suitable for read-heavy workloads.

Important Considerations

- Replication lag – slaves may temporarily have older data.
- Read-after-write consistency – if the latest data is required immediately after a write, read from the master or use appropriate consistency settings.
- Master failure – requires a failover mechanism to promote a new master.
- Slaves are generally read-only – write operations must be sent to the master.
- Replication can be synchronous or asynchronous – affects consistency and performance.

Conclusion

For a NoSQL database with very high read traffic and relatively few writes, Master-Slave Replication is an effective approach. Writes go to the master, while reads are distributed across multiple slaves, improving performance, scalability, and availability.

1. Non-Uniform Data Support

schemaless NoSQL database allows different records in the same collection
o have different fields, which is ideal for non-uniform data.

Product 1 (Laptop)

```
{
    "product_id": 101,
    "name": "Laptop",
    "brand": "Dell",
    "RAM": "16 GB",
    "storage": "512 GB SSD"
}
```

Product 2 (Shoes)

```
{
    "product_id": 102,
    "name": "Shoes",
    "brand": "Nike",
    "size": "9",
    "material": "Leather",
    "color": "Black"
}
```

Both products can be stored in the same collection even though they have different attributes.

2. Frequently Changing Custom Fields

New fields can be added easily without redesigning the database structure.

Initial Version

```
{
  "name": "Product",
  "price": 1000,
  "brand": "ABC"
}
```

Later Version

```
{
  "name": "Laptop",
  "price": 55000,
  "brand": "ABC",
  "RAM": "16 GB",
  "processor": "i7"
}
```

New Fields Added

```
{
  "name": "Laptop",
  "price": 55000,
  "brand": "ABC",
  "RAM": "16 GB",
  "processor": "i7",
  "rating": 4.5,
  "wireless_charging": tr
}
```

The application can evolve and add new fields as requirements change, without modifying existing data.

3. Flexible Data Model and User-Defined Fields

sers or different categories can store their own set of fields based on their needs.

Customer 1

```
{
  "name": "Ravi",
  "email": "ravi@example.com",
  "phone": "9876543210",
  "favorite_color": "Blue",
  "hobbies": ["Reading", "Photography"]
}
```

Customer 2

```
{
  "name": "Sita",
  "email": "sita@example.com",
  "linkedin": "profile",
  "skills": ["Python", "Java"],
  "certifications": ["AWS"]
}
```

A schemaless database can store both records without requiring a fixed set of columns, making it ideal for user-defined attributes.

4. Why Schemaless NoSQL is Suitable

Handles non-uniform data
Different records can have different fields.

Supports frequently changing fields
New attributes can be added easily.

Suitable for user-defined attributes
Users can store custom information.

Easy application evolution
Old and new records can coexist.

Good scalability
Works well for large and growing applications.

Conclusion

Yes, a schemaless NoSQL database is suitable.

A schemaless NoSQL database is a good choice when data is non-uniform and new custom fields are added frequently, because of its flexible schema, easy evolution and ability to store different attributes for different records.

is Sharding?

chnique used to divide a large dataset into smaller
rds and distribute those shards across multiple
s (nodes) in a NoSQL database.

stores a portion of the total data.

gether form the complete dataset.

is used to determine which shard should store a
of data.

lps in horizontal scalability, allowing the system to
amounts of data and high traffic.

ioned across multiple nodes.

hard) contains a subset of the data.

d and write performance by distributing the workload.

sed in NoSQL databases such as MongoDB, Cassandra,

harding Works (With Example)

t arrives at the application with a customer_id.

er uses the shard key (customer_id) to determine
ard contains the data.

est is sent only to the relevant shard.

d processes the read or write operation.

lt is returned to the application.

arios

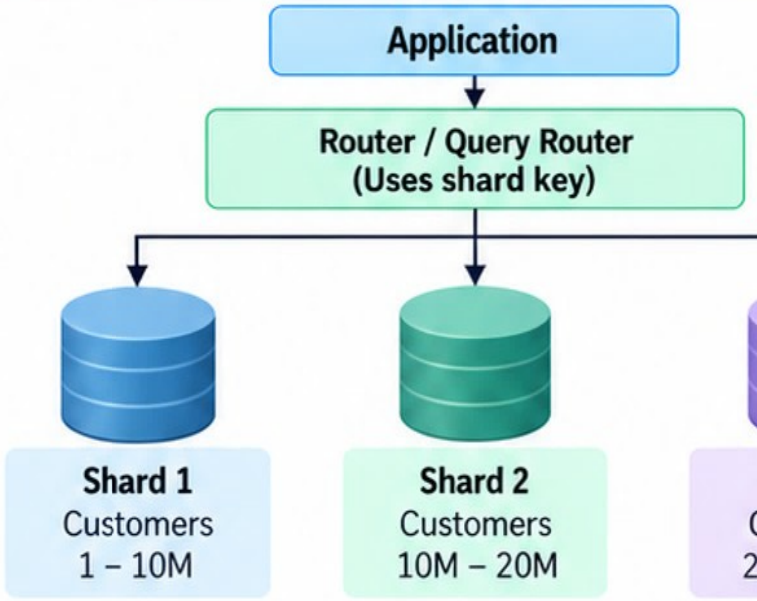
Customer ID	Target Shard	Action
5,000,123	Shard 1	Fetch from Shard 1
15,678,901	Shard 2	Fetch from Shard 2
25,432,110	Shard 3	Store in Shard 3

2. Example: E-commerce Ap

Consider an e-commerce application that stores custo
The database has 30 million customer records. We ca
based on the customer_id using range-based sharding

Data Distribution Plan

Shard	Customer ID Range	Rec
Shard 1	1 – 10,000,000	
Shard 2	10,000,001 – 20,000,000	
Shard 3	20,000,001 – 30,000,000	



4. Benefits and Consideration

Benefits of Sharding

- Improves read performance (queries distributed across shards).
- Improves write performance (writes distributed across shards).
- Supports horizontal scalability (add more shards as data grows).
- Reduces load on individual nodes.
- Allows handling of very large datasets.
- Helps achieve better resource utilization.

Considerati

- Choosing a important data distrib
- Cross-shar more comp additional
- Adding or may requir
- Operationa is higher c single-nod

Conclusion

Sharding is an effective technique in NoSQL databases to d
across multiple nodes, improving performance, scalability, a
In the e-commerce example, customer records are split into
on customer_id ranges, allowing the system to handle large volum

1. What is Sharding in NoSQL?

harding is a technique used to divide a large dataset into maller parts called shards and distribute those shards cross multiple database servers (nodes) in a NoSQL database.

Each shard stores a portion of the total data.

All shards together form the complete dataset.

A shard key is used to determine which shard should store a given piece of data.

Sharding enables horizontal scalability, allowing the system to handle large amounts of data and high traffic.

Key Benefits

- Distributes data across multiple nodes
- Improves read and write performance
- Supports large storage capacity
- Enables horizontal scaling
- Commonly used in NoSQL databases such as MongoDB, Cassandra, HBase

2. Why Sharding is Needed?

As the amount of data and user requests grow, a single server may become a performance bottleneck.

Without Sharding (Single Server)



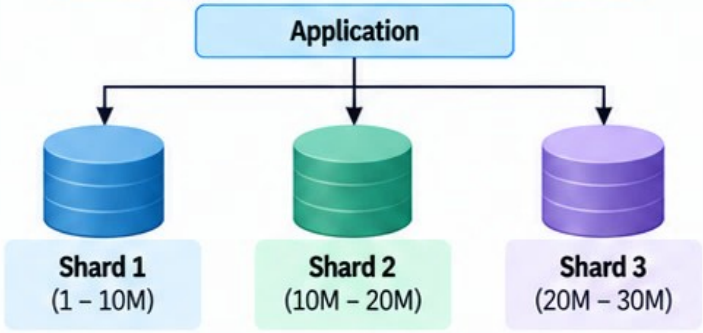
30 Million Customer Records

Problems:

- High CPU usage
- High memory usage
- Increased response time
- Limited storage capacity
- Cannot handle large traffic

With Sharding (Multiple Servers)

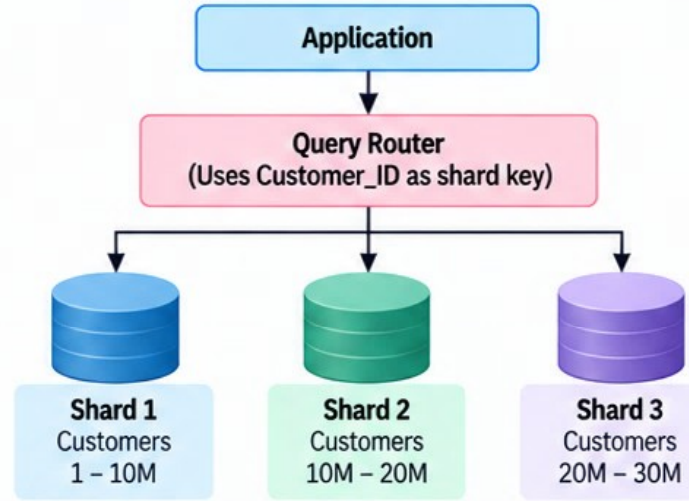
Data is divided and distributed across multiple servers.



3. Example: E-commerce Application

Consider an e-commerce application that stores 30 million customer records. We can shard the data based on Customer_ID using range-bas sharding.

Shard	Customer ID Range	Records (Approx.
Shard 1	1 – 10,000,000	10 million
Shard 2	10,000,001 – 20,000,000	10 million
Shard 3	20,000,001 – 30,000,000	10 million



4. How Sharding Works (Example Operations)

Operation	Customer_ID	Target Shard	Action
ad customer	5,000,123	Shard 1	Fetch from Shard 1
ad customer	15,678,901	Shard 2	Fetch from Shard 2
ert customer	25,432,110	Shard 3	Store in Shard 3

Flow of a Request

1. A request arrives at the application with a Customer_ID.
2. The router uses the shard key to determine the target shard.
3. The request is sent only to the relevant shard.
4. The shard processes the read or write operation.
5. The result is returned to the application.

5. Benefits, Types and Considerations

Benefits of Sharding

- Improves read performance (queries distributed across shards)
- Improves write performance (writes distributed across shards)
- Supports horizontal scalability
- Reduces load on individual nodes
- Allows handling of very large datasets
- Better resource utilization
- Increases overall system throughput

Types of Sharding

1. **Range-Based Sharding** (Data is divided based on ranges of the shard key)
2. **Hash-Based Sharding** (Uses a hash function on the shard key)
3. **Geographic Sharding** (Data is distributed based on geographic regions)

Considerations

- Choosing a good shard key is important to avoid uneven distribution.
- Cross-shard queries can be more complex and may require additional processi
- Adding or removing shards may require data rebalanci
- Operational complexity is higher compared to a sing node setup.
- Uneven data distribution can create hot shards.

1. What is Sharding?

harding is the process of dividing a large dataset into smaller parts called shards nd distributing those shards across multiple database servers (nodes). Each shard ontains a portion of the data, and the workload is distributed across the nodes.

Without Sharding

All data is stored on a single server.

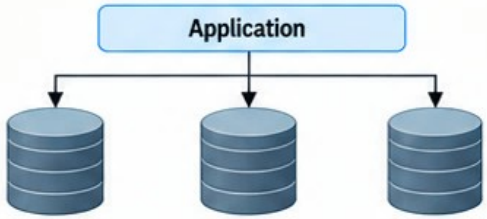


Server 1
(All Data)

- Handles all read requests
- Handles all write requests
- Becomes a performance bottleneck as data and traffic grow

With Sharding

Data is divided into multiple shards and stored on different servers.



Sharding enables horizontal scalability by distributing data and workload across multiple nodes.

2. How Sharding Improves Read Performance

Sharding improves read performance by allowing read requests to be distributed across multiple shards, so they can be processed in parallel.

Without Sharding

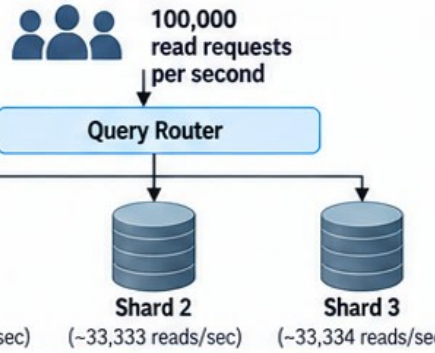
All read requests go to a single server.



High load
Slower response time

With Sharding

Read requests are distributed across multiple shards using a routing mechanism.



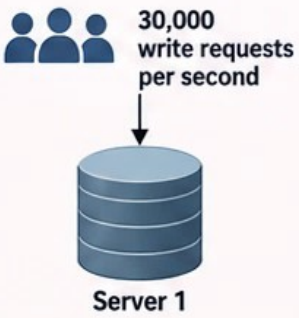
- Multiple shards process reads in parallel.
- Each node handles a smaller portion of the data.
- Improves overall read throughput and reduces response time.

3. How Sharding Improves Write Performance

harding **improves write performance** by distributing write operations across multiple shards, o that inserts, updates, and deletes are handled by different nodes.

Without Sharding

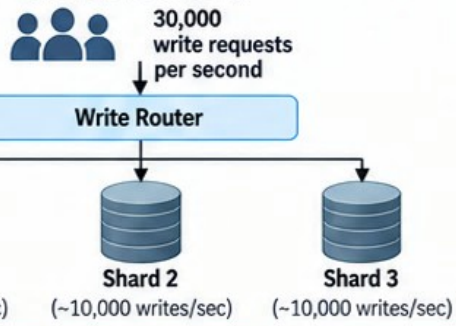
ll write requests go to a single server.



High load
Write bottleneck

With Sharding

Write requests are distributed to different shards based on the shard key.



- Write workload is spread across multiple nodes.
- Reduces CPU, disk and memory contention.
- Increases total write throughput.
- Supports higher data ingestion rates.

4. Example, Comparison and Key Takeaways

Example: E-commerce Customer Data

A database stores 30 million customer records.

Without Sharding



Server 1
Customers 1 – 30,000,000

With Sharding (Range-Based)



Key Takeaways

- Sharding divides data into smaller, manageable parts.
- Read requests are distributed across shards and processed in parallel.
- Write operations are also distributed, reducing the load on each node.
- Improves read and write throughput, supports large datasets, and enables horizontal scalability.
- Performance depends on choosing a good shard key and proper data distribution.
- Sharding can be used along with replication for better availability and read scaling.

Comparison

Aspect	Without Sharding	With Sharding
Data storage	Single node stores all data	Data distributed across multiple no
Read workload	Concentrated on one node	Distributed across shards
Write workload	Concentrated on one node	Distributed across shards
Performance	Limited by single node capacity	Higher throughput (parallel processi
Scalability	Vertical scaling only (limited)	Horizontal scaling (add more shards
Large datasets	Difficult to handle	Better suited
Query complexity	Simpler	Can be more com (cross-shard quer

Conclusion

Sharding improves both read and write performance by distribut data and workload across multiple nodes, enabling parallel proces and horizontal scalability.